



A fast algorithm for color space conversion and rounding error analysis based on fixed-point digital signal processors[☆]



Zhao-Guang Liu^{a,*}, Sheng-Yong Du^b, Yang Yang^c, Xiu-Hua Ji^a

^aShandong Provincial Key Laboratory of Digital Media Technology, School of Computer Science and Technology, Shandong University of Finance and Economics, Jinan, China

^bSchool of Management Science and Engineering, Shandong University of Finance and Economics, Jinan, China

^cSchool of Information Science and Engineering, Shandong University, Jinan, China

ARTICLE INFO

Article history:

Available online 14 February 2013

ABSTRACT

The YCbCr (luminance, chrominance-blue, and chrominance-red) color space is adopted in video codecs or transmission, while the HSV (hue, saturation and value) color space is used in some video analysis algorithms. In this paper, a fast algorithm based on fixed-point digital signal processors (DSPs) is proposed for YCbCr to HSV conversion. Floating-point multiplications are replaced with fixed-point shifts, 16-bit fixed-point multiplications, and additions. To compensate for rounding error and convert floating-point multiplication into 16-bit fixed-point multiplication in the calculation, tables occupying 225 bytes and 256 bytes are established, respectively. According to the experimental results, the error caused by the proposal is within the acceptable range. At the same time, the proposed algorithm is about 10 times faster than traditional YCbCr to HSV conversion algorithm on a fixed-point digital signal processor (DSP) platform and about 1.41 times faster on a personal computer (PC) platform.

© 2013 Elsevier Ltd. All rights reserved.

1. Introduction

A color image can be represented by multiple color space models, such as RGB (red, green, and blue), YCbCr (luminance, chrominance-blue, and chrominance-red) [1], and HSV (hue, saturation and value) [2,3]. RGB describes a color based on the three primary colors and light system theory. YCbCr describes a color in accordance with the principles of brightness and chromatic aberration. In YCbCr space, Y is the luminance component, Cr reflects the difference between the red component and the illumination value of the input RGB signal, and Cb reflects the difference between the blue component and the illumination value of the input RGB signal. In the HSV model, H, S, and V represent hue, saturation, and brightness, respectively. The V-axis of the HSV model corresponds to the main diagonal of the RGB color space.

Since RGB representations are highly correlated, they are not well suited to independent coding. Therefore, compression standards such as joint photographic experts group (JPEG), moving pictures experts group (MPEG), and H.264/AVC (AVC: advanced video codec) employ the YCbCr color space. At the same time, HSV is always used in video analysis; e.g., in fire or flame detection [4,5], skin color detection [6], and image segmentation [7]. In some applications, video is decoded from a bitstream directly, and the HSV color space is utilized for image or video analysis, so conversion from YCbCr to HSV is certainly necessary. Therefore, a fast conversion algorithm is helpful to reduce the computational complexity in such applications.

[☆] Reviews processed and recommended for publication to Editor-in-Chief by Guest Editor Dr. Jing Tian.

* Corresponding author.

E-mail address: liuzhg@sdufe.edu.cn (Z.-G. Liu).

Table 1

Color spaces of YCbCr and RGB.

	RGB			YCbCr		
	R	G	B	Y	Cb	Cr
Max	255	255	255	235	240	240
Min	0	0	0	16	16	16

There exists no direct conversion from YCbCr to HSV, and RGB variables are needed as intermediates. There are several different conversion standards in use. Without loss of generality, we focus on ITU-R Recommendation BT.601-5 [8] as well as Refs. [2,3]. According to the standard [8], the relationship between RGB and YCbCr is

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1.402 \\ 1 & -0.344 & -0.714 \\ 1 & 1.772 & 0 \end{bmatrix} \times \begin{bmatrix} Y \\ Cb - 128 \\ Cr - 128 \end{bmatrix}. \quad (1)$$

The only difference between YCbCr and YUV (luminance and chrominance) is the direct component (DC) component of 128, and the conversion matrix is the same, so we focus on YCbCr below. According to the standard [8], the range of each component of YCbCr or RGB is that listed in Table 1.

According to Refs. [2,3], the formula for RGB to HSV conversion is

$$\begin{cases} H = 60 * \begin{cases} (G - B)/\delta, & \text{if } R = \max(R, G, B) \\ 2 + (B - R)/\delta, & \text{if } G = \max(R, G, B) \\ 4 + (R - B)/\delta, & \text{if } B = \max(R, G, B) \end{cases} \\ S = \delta / \max(R, G, B) \\ V = \max(R, G, B) \\ \text{where } \delta = \max(R, G, B) - \min(R, G, B). \end{cases} \quad (2)$$

The rest of this paper is organized as follows. Section 2 gives a detailed introduction to related work. Section 3 proposes a fast algorithm for YCbCr to HSV conversion. Section 4 presents experimental results based on a fixed-point DSP and a general-purpose PC. Finally, Section 5 states the conclusions.

2. Related work

As far as we know, there is currently no fast algorithm for conversion from RGB to HSV. There are several conventional methods for conversion from YCbCr to RGB, including hardware and software methods. Based on comparisons of those methods, this study prescribes a way to find the most economical expression at given quantification error, which is more appropriate for fixed-point DSPs.

2.1. Lookup table method

The lookup table (LUT) method is a highly efficient method, especially for embedded systems [9]. There are two kinds of LUT methods for YCbCr to RGB conversion; i.e., one-check LUTs and two-check LUTs. For the 24-bit case, the one-check LUT needs no calculation and simply checks the index table of $6 \times 256 \times 256 \times 256 = 100663296$ bytes long, which becomes impractical for embedded systems. For the two-check LUT, there are two series corresponding to the upper and lower 4 bits of YCbCr and RGB data, respectively, and there are six tables in each series, so the index table has only $2 \times 6 \times 16 \times 16 \times 16 = 49152$ bytes. The 8-bit data are first broken into two sets of 4-bit data, the two index tables are then checked, and the results are added to obtain the final result. Compared with a one-check LUT, a two-check LUT needs extra blocking, shift, and addition operations, so a two-check LUT is relatively slow but occupies much less memory. The drawback of any LUT is the great amount of memory occupied.

2.2. Shifting method

In our previous work [10], a fast algorithm for YCbCr to RGB conversion on the fixed-point DSP was presented. Within the allowable range of error, the floating-point operations can be converted into fixed-point shift and addition operations, thus the computation speed was improved dramatically. In this paper, we will further analyze the shifting method and its rounding error.

Table 2
Algorithms in experiments.

YCbCr to RGB	RGB to HSV	YCbCr to HSV
Direct method [8]	Direct method [2]	Direct method [8,2]
Two-check LUT [20]	Fast method	LUT [20]+Fast
Scaling method		Scaling + Fast
Shifting method		Shifting + Fast
Yang method [10]		Yang [10]+Fast

2.3. Hardware method

This method uses an application-specific integrated circuit (ASIC) [11,12], field-programmable gate array (FPGA) [13–15], or operational amplifier and resistance network to realize the YCbCr to RGB conversion. Compared with software algorithms, a hardware method is rather fast in most cases since it only needs 1 or 2 clock cycles. The disadvantage is that a hardware method is not as convenient as a software algorithm.

2.4. Other methods

Ref. [16] proposed a joint implementation algorithm for computing color space conversions, quantizations, and discrete cosine transforms (DCTs). In fact, the YCbCr to RGB conversion embedded in the joint implementation is not analyzed specially. In Refs. [17,18], floating-point multiplication was split into multiplications of several integer matrices. However, too many matrices were introduced, accounting for the increased number of clock cycles and leading to high computational complexity.

The RGB variables are needed as intermediates during the conversion from YCbCr to HSV, which proceeds from YCbCr to RGB and from RGB to HSV. This paper presents a fast algorithm that can convert from the YCbCr color space to HSV color space. Fixed-point shifts, 16-bit fixed-point multiplications, additions, and an LUT are used to avoid unnecessary floating-point operations, thus the computational complexity is substantially reduced.

3. Proposal of fast algorithm for YCbCr to HSV conversion

In our previous work [10], a shifting method to achieve YCbCr to RGB conversion was presented and there were quantification errors in the R , G , and B components. For instance, the bit numbers for the R , G , and B components are 5, 6, and 5, respectively, in some commonly used 16-bit liquid crystal displays (LCDs). Therefore, the quantification errors are 3 bits for R , 2 bits for G , and 3 bits for B ; namely, the permitted quantification step is 8 for R , 4 for G , and 8 for B . In the present research, the errors in this shifting method are further analyzed. An error compensation table is proposed to compensate for errors in conversion. At the same time, a scaling method is proposed, and the rounding errors are analyzed in general. As with the shifting method, a table is proposed to compensate for rounding errors in conversion. With either the shifting method or the scaling method, the left maximal error in conversion is only 1, and this is beneficial for RGB to HSV conversion. Furthermore, a scaling table is established in RGB to HSV conversion, and the rounding error of the H component is zero when H is viewed as an integer value. In the whole conversion process, the maximal error in YCbCr to RGB conversion is only 1, and the maximal error in the H component is 3. Therefore, in some applications [5,6], the utilization of color components sometimes has scopes. For instance, in Ref. [6], the skin color was defined as $0 < H < 50$. The H component can certainly be viewed as an integer in this application. If we scale the S component with an integer, the scope $0.2 < S < 0.68$ can also be implemented with integers. Therefore, the conversion precision of the proposed algorithm is high enough for these applications. Since floating-point operations are converted into 16-bit fixed-point multiplications, fixed-point shifts, and additions in the proposal, the computational complexity can be greatly reduced.

The whole conversion process can be seen in Fig. 1. The proposal consists in two steps, which are YCbCr to RGB conversion and RGB to HSV conversion. In the YCbCr to RGB conversion, on the basis of our previous work [10], this paper will analyze rounding errors in the shifting method and the scaling method. Furthermore, floating-point operations are converted into 16-bit fixed-point multiplication, fixed-point shift, and addition operations. For compensation of rounding error in this step, an LUT method is also proposed. In the RGB to HSV conversion, because the range of data in multiplications is limited to [1,225], a fast method with a scaling table similar to the error compensation table is proposed.

According to Eq. (2), the calculation of the H component is based on the difference between two of the R , G , and B components. Since the Y component is included in the R , G , and B components simultaneously according to Eq. (1), the calculation of the H component does not involve the Y component. Moreover, the calculation of the H component is the most time-consuming part. Therefore, in the proposed algorithm, R , G , and B are replaced by the intermediate variables R' , G' , and B' , which do not contain the Y component. While not affecting the conversion result, two additions are saved. According to this analysis, Eqs. (1) and (2) can be written as follows:

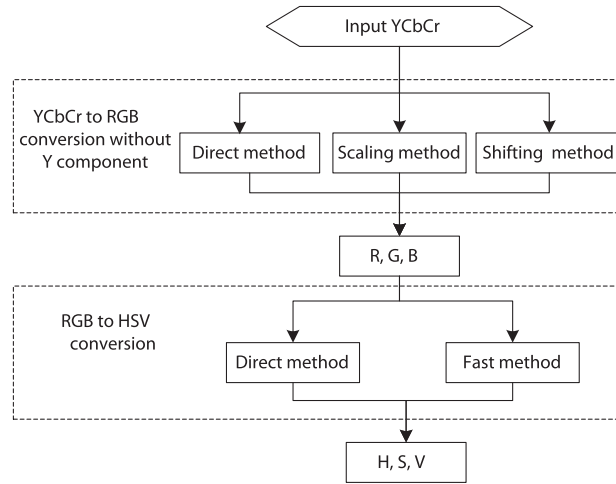


Fig. 1. Whole conversion process.

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = \begin{bmatrix} 0 & 1.402 \\ -0.344 & -0.714 \\ 1.772 & 0 \end{bmatrix} \times \begin{bmatrix} Cb - 128 \\ Cr - 128 \end{bmatrix} \quad (3)$$

$$\begin{cases} H = 60 * \begin{cases} (G' - B')/\delta, & \text{if } R' = \max(R', G', B') \\ 2 + (B' - R')/\delta, & \text{if } G' = \max(R', G', B') \\ 4 + (R' - B')/\delta, & \text{if } B' = \max(R', G', B') \end{cases} \\ V = \max(R', G', B') + Y \\ S = \delta / V \\ \text{where } \delta = \max(R', G', B') - \min(R', G', B'). \end{cases} \quad (4)$$

3.1. YCbCr to RGB conversion algorithm

In this section, we analyze rounding errors in the scaling method and shifting method [10], and we propose an LUT error compensation method. The calculation of the R' component is given as an example of the analysis.

3.1.1. Scaling method

According to Eq. (3), the calculation of R' component can be approximated as

$$R' = (1.402 * 2^M * Cr') >> M, \quad (5)$$

where M is an integer and $Cr' = Cr - 128$.

To determine the value of M , let us calculate the difference between Eqs. (3) and (5):

$$|\Delta R'| = |1.402 * Cr' - [(1.402 * 2^M * Cr') >> M]| \quad (6)$$

Obviously, the criterion for determining the value of M is finding the minimum value of $|\Delta R'|$, and the solution can be written formally as the argument M that minimizes $|\Delta R'|$:

$$\hat{M} = \arg \min_M |\Delta R'|. \quad (7)$$

According to the standard [8], the range of the Cr component is [16, 225], and the range of Cr' is [-112, 112]. In this paper, we determine the value of $\hat{M}$ by testing all values of Cr' according to Eq. (7).

Similar to the calculation of the R' component, the calculations of the G' and B' components can be scaled into integer multiplications, and the final formulas are

$$\begin{cases} R' = (Cr' \times 11485) >> 13 \\ G' = (-Cb' \times 2819 - Cr' \times 5850) >> 13. \\ B' = Cb' \times 14516 >> 13 \end{cases} \quad (8)$$

Since the range of Cr' is [-112, 112], the multiplications in Eq. (8) are all 16-bit fixed-point multiplications that can each be finished in one clock cycle on a DSP.

According to our experimental results, the maximal values of the rounding errors of R' , G' , and B' are 1, 2, and 1, respectively. A table is established in this paper to compensate for these errors. The rounding errors of R' and B' can be completely compensated for by using this LUT method, and the required memory size is only 225 bytes (the range of Cr' or Cb'). However, if the error of the G' component is completely compensated for by the LUT method, the required memory size is $225 \times 225 = 50625$ bytes, which is difficult for a DSP. Therefore, the errors caused by Cb' and Cr' are compensated for in the calculation of the G' component.

In the final implementation, the R' , G' , and B' components are calculated as

$$\begin{cases} R' = R' + (ErrTab[Cr - 16] \& 0x03) \\ G' = G' - ((ErrTab[Cb - 16] \& 0x0C) >> 2) - ((ErrTab[Cr - 16] \& 0x30) >> 4), \\ B' = B' + ((ErrTab[Cb - 16] \& 0xC0) >> 6) \end{cases} \quad (9)$$

where *ErrTab* is the error compensation table and the total memory size required is 225 bytes. The values in *ErrTab* are calculated in advance by testing all values of Cb' and Cr' . The rounding errors of the R' and B' components can be avoided by Eq. (9), and the maximal value of the rounding error of the G' component is only 1 according to our experimental results.

3.1.2. Shifting method

In our previous work [10], the shifting method was proposed on the basis of fixed-point DSPs. However, the rounding error in conversion was uncompensated. According to Eq. (3), YCbCr to RGB conversion needs to use floating-point calculations even though the result is an integer, while a floating-point calculation on a DSP is much slower than a fixed-point shift operation. In the floating-point operations, a division by 2^n can be replaced with a shift right by n bits. For example, a division by 2 can be replaced by a 1-bit shift to the right. Therefore, with the precision retained, floating-point multiplication can be replaced by the shift operation, which can significantly improve the processing speed.

Eq. (3) can be rewritten as follows:

$$\begin{cases} R' = a * Cr' \\ G' = -b_1 * Cb' - b_2 * Cr' \\ B' = c * Cb' \end{cases} \quad (10)$$

where a , b_1 , b_2 , c are coefficients to be determined, while $Cb' = Cb - 128$ and $Cr' = Cr - 128$.

To determine the coefficient a , let us calculate the difference between Eqs. (3) and (10):

$$|\Delta R'| = |(1.402 - a) * Cr'|. \quad (11)$$

Whatever the value of a , rounding error between Eqs. (11) and (3) exists because of truncation error in the shift operation. For example, when $Cr' = 11$ and $a = 1.40625 = 1 + (1 >> 2) + (1 >> 3) + (1 >> 5)$, $|\Delta R'| = 0.04675$ according to Eq. (11). In fact, the calculation of the R' component by the shift operation is $11 + (11 >> 2) + (11 > 3) + (11 >> 5) = 14$, while $1.402 \times 11 = 15.4$. Therefore, the rounding error is $15.4 - 14 = 1.4$, which is much greater than 0.04675. On the other hand, one floating-point multiplication is replaced by several shift operations, and the increase in the number of shift operations leads to a higher computational complexity. Considering the effects of rounding error and computational complexity simultaneously, the YCbCr to RGB conversion with fixed-point shift and addition operations is

$$\begin{cases} R' = Cr' + \frac{Cr'}{2^2} + \frac{Cr'}{2^3} + \frac{Cr'}{2^5} \\ G' = -(\frac{Cb'}{2^2} + \frac{Cb'}{2^4} + \frac{Cb'}{2^5}) - (Cr' - \frac{Cr'}{2^2} - \frac{Cr'}{2^5} - \frac{Cr'}{2^7}) \\ B' = Cb' + \frac{Cb'}{2} + \frac{Cb'}{2^2} + \frac{Cb'}{2^6} \end{cases} \quad (12)$$

After testing all values of Y , Cb' , and Cr' , we determine the rounding errors of R' , G' , and B' , which are 2, 4, and 2, respectively. A table similar to that of the scaling method is established to compensate for the rounding errors, and the required memory size is also 225 bytes.

3.2. RGB to HSV conversion algorithm

Consistent with the premise of not affecting the result, the 16-bit fixed-point multiplication and shift operation can also be used to simplify the RGB to HSV conversion. In the formula for RGB to HSV conversion, division is used when calculating the H component and S component. Division is not provided by most fixed-point DSPs and is always implemented by iterative subtraction, which leads to a low speed.

This paper uses an LUT to implement the division. In the calculation of the H component, the divisor $\delta = \max(R', G', B') - \min(R', G', B')$ has a range of $[1, 255]$, thus the reciprocal of the divisor, $1/\delta$, can be put into an array *ScaleTab*. In order to use 16-bit fixed-point multiplication in fixed-point DSPs, the values in *ScaleTab* are converted into integers by a scale factor. Finally, the resultant H component should be divided by the scale factor (implemented by the shift operation). In the calculation of the S component, the divisor $\max(R', G', B') + Y$ has a range of $[1, 255]$, which is the same as that of δ , so the divisor can share the values in *ScaleTab*. Based on the above discussion, the *ScaleTab* array is established as

$$\text{ScaleTab}[\text{delta} - 1] = (\text{int}) \left\lceil \frac{\text{scalor}}{\text{delta}} + 0.5 \right\rceil, \quad (\text{delta} = 1, 2, \dots, 254, 255), \quad (13)$$

where $\text{scalor} = 2^N$, with N an integer to be determined, and (int) denotes a rounding operation.

Let us analyze the rounding error by means of the following equation, since the process is similar when R' or B' is the maximal value:

$$H = 120 + \frac{60 * (B' - R')}{\text{delta}}, \text{ if } G' = \max(R', G', B'). \quad (14)$$

The calculation of H' is implemented as

$$H' = 120 + \frac{(60 * (B - R) * \text{ScaleTab}[\text{delta} - 1])}{\text{scalor}}. \quad (15)$$

The rounding error between Eqs. (14) and (15) is

$$|\Delta H| = |H - H'| = \left| \frac{60 * (B' - R')}{\text{delta}} - \frac{(60 * (B' - R') * \text{ScaleTab}[\text{delta} - 1])}{\text{scalor}} \right| < \text{Err} \quad (16)$$

Eq. (16) can be rewritten as

$$|\text{scalor} - \text{delta} * \text{ScaleTab}[\text{delta} - 1]| < \left| \frac{\text{scalor} * \text{delta} * \text{Err}}{60 * (B' - R')} \right| \quad (17)$$

Because $|\text{delta}| = |\max(R', G', B') - \min(R', G', B')| \geq |B' - R'|$, the formula can be simplified to

$$|\text{scalor} - \text{delta} * \text{ScaleTab}[\text{delta} - 1]| < \left| \frac{\text{scalor} * \text{Err}}{60} \right|. \quad (18)$$

We can use Eq. (13) to create a lookup table; i.e., a *ScaleTab* whose elements satisfy Eq. (18). The parameter *Err* is used to control the maximal error between H and H' . According to Eqs. (13) and (14), the calculation of the H component is

$$H = 120 + ((60 * (B' - R') * \text{ScaleTab}[\text{delta} - 1]) >> N). \quad (19)$$

Because the values in the *ScaleTab* array have the form $2^N/\text{delta}$, the calculated result should be shifted right by N bits.

The range of the S component in HSV color space is $[0, 1]$, and this range can easily be scaled into $[0, K]$ for the purpose of satisfying the requirements of applications or algorithms. For example, when $K = \text{scalor}$, the S component is calculated as follows:

$$S = (\text{scalor} * \text{delta} * \text{ScaleTab}[V]) >> N \quad (20)$$

4. Experimental results

Experiments based on a DSP and a general-purpose PC were performed to evaluate the efficiency of the proposed algorithm. The DSP platform is the TI DaVinci DM6446 [19], which is a DSP platform widely used in multimedia signal processing. The PC platform includes an Intel Core processor, with a CPU frequency of 3.2 GHz, and 3 GB of memory. The experiments were performed on a sequence named Foreman, and the frame size was that of the common intermediate format (CIF) (352×288 pixels). A single frame is converted on the DSP platform; i.e., the number of pixel is $352 \times 288 = 101376$. The numbers of clock cycles required are listed in Tables 3–5. Since the time consumed in converting a single frame on the PC is too slight for evaluating the different algorithms, 300 frames are converted on the PC; i.e., the number of pixels is $300 \times 352 \times 288 = 30412800$. We performed the experiments using the C language, and the programs can easily be migrated to other platforms.

The conversions involved in this paper include YCbCr to RGB, RGB to HSV, and YCbCr to HSV. Table 2 lists the different algorithms for these conversions. The direct method is the direct use of the original conversion formulas, Eqs. (1) and (2), in YCbCr to RGB and RGB to HSV conversions, respectively. In YCbCr to HSV conversion, the direct method is a cascaded execution of Eqs. (1) and (2). In YCbCr to RGB conversion, the scaling method and the shifting method are the two YCbCr to RGB conversions proposed in Section 3.1. We also compare the proposed YCbCr to RGB conversion with the two-check LUT [20] method and the method in our previous work [10] (herein called the Yang method). In RGB to HSV conversion, the fast method

Table 3
Comparison of computational complexities of YCbCr to RGB algorithms.

Platforms	Direct method	Two-check LUT	Scaling method	Shifting method	Yang method
DSP (clock)	98292495	525568	625804	805238	786920
PC (ms)	672	125	313	422	407

Table 4
Comparison of computational complexities of RGB to HSV algorithms.

Platforms	Direct method	Fast method
DSP (clock)	68486934	13957237
PC (ms)	813	687

Table 5
Comparison of computational complexities of YCbCr to HSV algorithms.

Platforms	Direct method	LUT + Fast	Scaling + Fast	Shifting + Fast	Yang + Fast
DSP (clock)	181510967	16604640	17014844	17691975	17382219
PC (ms)	1157	662	733	929	903

od is the algorithm proposed in Section 3.2. In YCbCr to HSV conversion, different YCbCr to RGB algorithms are combined with our proposed RGB to HSV method; e.g., LUT + Fast means the two-check LUT and the fast method.

4.1. Comparison of computational complexities

The computational complexities of different algorithms in conversion are shown Tables 3–5.

According to our experimental results (see Table 3), in YCbCr to RGB conversion the LUT method achieves the best result. Compared to the direct method, the LUT method is about 187 times faster on the DSP and 5.4 times faster on the PC. As mentioned above, the crucial disadvantage of the LUT is the great amount of memory consumed. The scaling method performs better than the shifting method not only on the DSP but also on the PC. For example, compared to the direct method, the scaling method and the shifting method speed up the conversion on the DSP by factors of about 157 and 122, respectively.

According to Eqs. (8), (9), and (12), the conversion of a single pixel by the shifting method requires 4 16-bit fixed-point multiplications, 9 fixed-point shifts, 6 additions, and 4 logical operations, while the same conversion by the scaling method requires 16 fixed-point shifts, 21 additions, and 4 logical operations. Each of these operations can be finished in one clock cycle on the DSP. Though 16-bit fixed-point multiplication is not needed in the shifting method, the numbers of additions and fixed-point shifts are greater than in the scaling method. Hence, the scaling method performed better than the shifting method in our experiments. In most DSPs, there are more shifters than multipliers. Therefore, the disadvantage of the shifting method can be eliminated by using the hardware in the DSP sufficiently through assembly language or intrinsic functions provided by the manufacturer.

Compared to the shifting method, the Yang method omits three additional operations. Therefore, the shifting method is slightly slower than the Yang method. However, according to the experimental results listed in Table 7, the conversion accuracy of the shifting method is better than that of the Yang method.

For RGB to HSV conversion, the times consumed by the direct method and our proposed algorithm are listed in Table 4. In contrast with Eq. (2), the floating-point multiplications by $1/\delta$ and $1/\max(R, G, B)$ are converted into fixed-point shift operations and table lookups. There are no improvements in the other calculations; e.g., to find the maximums of R , G , and B . Therefore, the effectiveness of the reduction in computational complexity is not as great as for YCbCr to RGB conversion. Compared to the direct method, our proposed algorithm speeds up the conversion by factors of about 4.9 and 1.2 on the DSP and PC, respectively.

Table 6
Comparison of accuracies for H component on PC.

Algorithms	$E < 1$	$1 \leq E < 2$	$2 \leq E \leq 3$
Scaling method	10480120 (94.10%)	657348 (5.90%)	32 (0.00028%)
Shifting method	10985335 (98.63%)	132764 (1.19%)	19401 (0.17%)

Table 7
Comparison of accuracies for RGB components on PC.

Test sequence	$E = 0$ (%)			$E = 1$ (%)			$E = 2$ (%)		
	Yang	Shift	Scale	Yang	Shift	Scale	Yang	Shift	Scale
Foreman	97.000	97.233	97.001	2.9838	2.767	2.999	0.0156	0	0
Hall	95.915	95.895	95.901	4.0821	4.105	4.099	0.0034	0	0
Container	97.730	97.920	98.192	2.2583	2.08	1.808	0.0117	0	0

The fast YCbCr to HSV conversion algorithms are combinations of the YCbCr to RGB conversion algorithms with the fast RGB to HSV conversion algorithm. The experimental results are listed in Table 5. According to the results, the different fast algorithms are close to each other in terms of reduction in computational complexity; e.g., LUT + Fast, Scale + Fast, and Shift + Fast speed up the conversion on the DSP by factors of 10.9, 10.7, and 10, respectively. According to the results in Tables 3 and 4, the times consumed by the fast algorithms are much lower in YCbCr to RGB conversion than in RGB to HSV conversion. That is, in YCbCr to HSV conversion, the percentage of time consumption is much greater for RGB to HSV conversion than for YCbCr to RGB conversion. This is why the three fast algorithms perform similarly in terms of computational complexity.

4.2. Comparison of conversion accuracies

In addition to the comparison of computational complexities, we also verified the accuracy of the results. We tested all values of Y , Cb , and Cr in the ranges $[16, 235]$, $[16, 240]$, and $[16, 240]$, according to the standard [8]. Therefore, the total number of pixels was $220 \times 225 \times 225 = 11137500$. The symbol E in Table 6 represents the difference in the H component between Eq. (2) and the proposal. For example, the number of pixels calculated by the scaling method and satisfying $E < 1$ was 10480120, so the accuracy was $10480120 / 11137500 = 94.10\%$.

Since we completely compensated for the rounding errors of the R and B components in the scaling method and the shifting method, the error shown in Table 6 is caused by the rounding error of the G component in the YCbCr to RGB conversion. Based on the experimental results in Table 6, the following conclusions can be drawn: When $E < 1$, the accuracy of the shifting method (98.63%) is greater than that of the scaling method (94.10%). Conversely, when $1 \leq E \leq 2$, the scaling method performs better than the shifting method. Overall, the maximal error caused by the proposal is 3.

In license plate recognition, fire detection, and other video analyses, the judgment of color often concerns whether a pixel is located in some sectors that meet a criterion, such as for license plate color, flame color, or skin color. Therefore, the error introduced by the proposed algorithm is perfectly acceptable in such applications and the proposal can be widely adopted in these video analyses.

We also compared the conversion accuracy of this research with that of work using the Yang method. The results are shown in Table 7. The symbol E in Table 7 represents the difference in the R , G , and B components between Eq. (1) and the proposal. Based on Table 7, a conclusion can be drawn: that is, the conversion accuracy can obviously be improved by not only the shifting method but also the scaling method, when comparing these with our previous work (the Yang method).

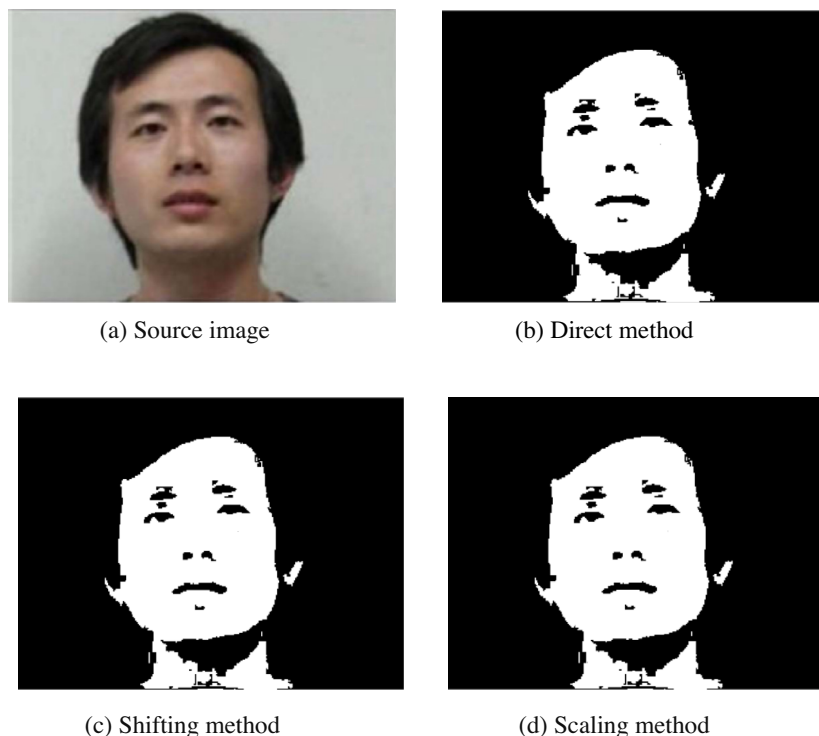


Fig. 2. Skin color detected by method proposed in Ref. [6] with different conversion algorithms. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4.3. Comparison in applications

In our experiments, two applications utilizing the HSV color space were implemented [5,6]. For a test of skin color detection [6], a face dataset was downloaded [21]. For a test of flame detection, color images with flames were downloaded from the Internet. Some detection results are shown in Figs. 2 and 3. Moreover, we compared the classification accuracies and computational complexities of the algorithms in these two applications, and the results are listed in Tables 8 and 9. In the comparison of classification accuracies, the accuracy of 'Direct Method' is assumed as 100%, just as shown in Table 9. The frame size in skin color detection was 640×480 , and the total number of frames was 200. The frame size in flame detection was 240×160 , and the total number of frames was 100. In our experiments, a single frame was processed on the DSP platform and every frame was processed on the PC platform.

According to the detection results shown in Figs. 2 and 3, there are no obvious differences among the conversion methods. That is, the conversion accuracies of the proposed algorithm are sufficient for such applications. Furthermore, according to the experimental results in Table 8, the computational complexity in skin detection and flame detection can be greatly reduced on the DSP. With our proposal, the computational complexity can also be reduced by about 20% on the PC.

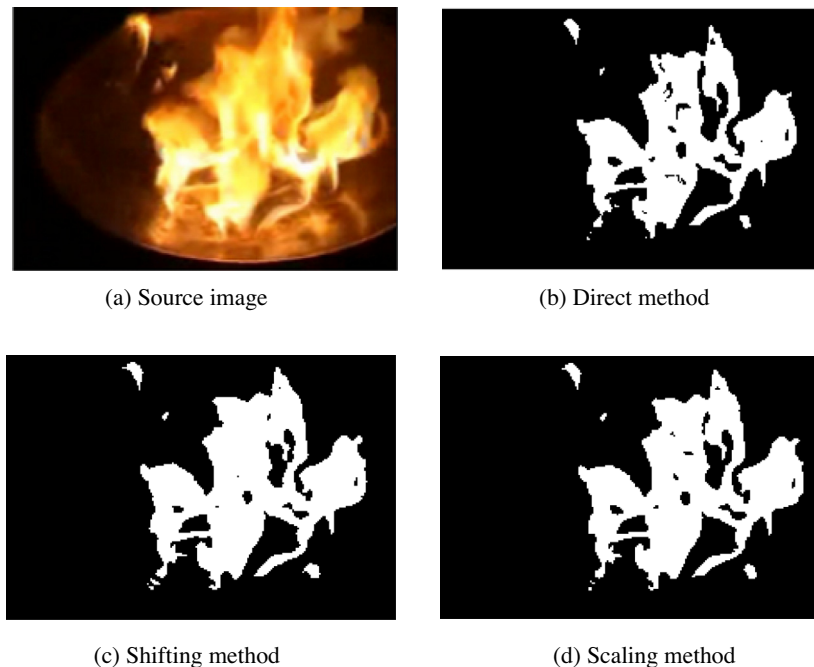


Fig. 3. Flame color detected by method proposed in Ref. [5] with different conversion algorithms. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 8

Comparison of computational complexities of different conversion algorithms.

Applications		Direct method	Fast method
Skin detection	DSP (clock)	91248600	27758852
	PC (ms)	2610	2109
Flame detection	DSP (clock)	76771719	25496760
	PC (ms)	2417	1998

Table 9

Comparison of classification accuracy of different conversion algorithms.

Applications	Direct method (%)	Shifting method (%)	Scaling method (%)
Skin detection	100	99.89	99.36
Flame detection	100	94.39	95.85

5. Conclusion

In some video analysis applications with high demands in real time, e.g. skin color detection, fire detection, the YCbCr color space to the HSV color space conversion is needed. This paper has proposed a fast algorithm for implementing the YCbCr to HSV conversion based on fixed-point DSPs. The proposal uses 16-bit fixed-point multiplications, fixed-point shifts, and additions instead of floating-point multiplications. After the analysis of rounding error in YCbCr to RGB conversion, a lookup table method was proposed to compensate for errors in the calculations, and the required memory size is 225 bytes. With the proposed error compensation method, the rounding errors in the *R* and *B* components can be avoided, and the maximal error in the *G* component is only 1. Furthermore, the *Y* component is removed from our YCbCr to HSV conversion to reduce the computational complexity. To convert floating-point multiplication to 16-bit fixed-point multiplication in RGB to HSV conversion, a lookup table method was proposed in which the memory size is only 256 bytes. Finally, experimental results show that the proposed algorithms are much faster than the traditional algorithms on either a DSP or a general-purpose PC, while the error introduced is within the acceptable range, so the proposal has good applicability to video analysis systems.

Acknowledgments

This research is supported by the National Natural Science Foundation of China (Nos. 61103118, 61073162, and 61203269), the Shangdong Young Scientists Award Fund (No. BS2012DX027), and the Shandong Natural Science Foundation of China (Nos. ZR2011FQ021 and ZR2009GL013).

References

- [1] Poynton CA. Digital video and HDTV: algorithms and interfaces. San Francisco, Amsterdam, Boston: Morgan Kaufmann Publishers; 2003. p. 313–21.
- [2] Smith AR. Color gamut transform pairs. *Comput Graph* 1978;12(3):12–9.
- [3] Chen W, Shi YQ, Xuan G. Identifying computer graphics using HSV color model and statistical moments of characteristic functions. In: *Proc IEEE Conf Multimedia and Expo*, Beijing, China; 2007. p. 1123–6.
- [4] Zhang D, Rao Y, Zhao JH, Zhao JM, Hu A, Cai B. Feature based segmentation and clustering on forest fire video. In: *Proc IEEE Conf Robotics and Biomimetics*; 2007. p. 1788–92.
- [5] Wang SJ, Jeng DL, Tsai MT. Early fire detection method in video for vessels. *J Syst Softw* 2009;82:656–67.
- [6] Zhang XN, Jiang J, Liang ZH, Liu CL. Skin color enhancement based on favorite skin color in HSV color space. *IEEE Trans Consum Electr* 2010;56(3):1789–93.
- [7] Chen TW, Chen YL, Chien SY. Fast image segmentation based on *K*-means clustering with histograms in HSV color space. In: *Proc IEEE Workshop Multimedia Signal Processing*, Queensland, Australia; 2008. p. 322–5.
- [8] ITU-R recommendation BT.601-5: studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios.
- [9] Webb JH. Efficient table access for reversible variable-length decoding. *IEEE Trans Circ Syst Video Technol* 2001;11(8):981–5.
- [10] Yang Y, Yuhua P, Zhaoguang L. A fast algorithm for YCbCr to RGB conversion. *IEEE Trans Consum Electr* 2007;53(4):1490–3.
- [11] Wang R. VLSI design of universal color conversion circuit. In: 2004 Asia-Pacific radio science conference, Qingdao, China; 2004. p. 269–72.
- [12] Bilal M, Masud S. Efficient color space conversion using custom instruction in a RISC processor. In: *Proc IEEE symp circuits and systems*, New Orleans, USA; 2007. p. 1109–12.
- [13] Sapkal AM, Munot M, Joshi MA. $R'G'B'$ to $Y'CbCr$ color space conversion using FPGA. In: *IET conf wireless, mobile and multimedia networks*, grand, Mumbai, India; 2008. p. 255–8.
- [14] Bensaali F, Amira A. Design and efficient FPGA implementation of an RGB to YCrCb color space converter using distributed arithmetic. *Springer Lecture Notes in Computer Science*, vol. 12; 2004. p. 991–5.
- [15] Ahirwal B, Khadtare M, Mehta R. FPGA based system for color space transformation RGB to YIQ and YCbCr. In: *Proc IEEE Conf Intelligent and Advanced Systems*, Kuala Lumpur, Malaysia; 2007. p. 1345–9.
- [16] Ji XH, Zhang CM, Zhang XF. An efficient joint implementation of three stages for fast computation of color space conversation in image coding/decoding. In: *International Journal of Multimedia Tools and Applications*, Online First; October 2011.
- [17] Pei SC, Ding JJ. Reversible integer color transform. *IEEE Trans Image Process* 2007;16(6):1686–91.
- [18] Pei SC, Ding JJ. Reversible integer color transform with bit-constraint. In: *Proc IEEE Conf Image Processing*, Genoa, Italy; 2005. p. III-964–967.
- [19] TEXAS Instruments. TMS320DM6446 digital media system-on-chip. TEXAS: TEXAS instruments; 2007. p. 21–3.
- [20] Gillespy T, Rowberg AH. Dual lookup table algorithm: an enhanced method of displaying 16-bit gray-scale images to 8-bit RGB graphic systems. *J Digit Imaging* 1994;7(1):13–7.
- [21] The IMM Face Dataset is publicly. <<http://www2.imm.dtu.dk/~aam/>>.

Zhao-Guang Liu received the B.E. and M.S. degrees from Xi'an University of technology in 1998 and 2001 respectively, and the Ph.D. degree in School of Information Science and Engineering from Shandong University in 2009. He is now an associate professor at Shandong University of Finance and Economics. His current research interests include video analysis and digital image processing.

Sheng-Yong Du received M.S. degrees in School of Computer Science and Technology from Shandong University in 2006. He is now a lecturer at Shandong University of Finance and Economics. His current research interests include video analysis and digital image processing.

Yang Yang received the B.E. and Ph.D degree in 2004 and 2009 from Shandong University respectively. He has been a lecturer of the School of Information Science and Engineering, Shandong University since 2009. His current research interests include image analysis, video tracking and signal processing.

Xiu-Hua Ji received the B.E. and M.S. degrees in electronics from Shandong University in 1985 and 1988, respectively, and the Ph.D. degree in computer science and technology from Shandong University, Jinan, China, in 2009. She is now a professor at Shandong University of Finance and Economics. Her current research interests include image process and compression.